

Naive Bayes Classifier

📅 Start Date	@29 December 2025
☰ Weeks	Week 9

Notes are from reading "Naive Bayes and Text Classification" by Sebastian Raschka + GeekforGeeks

Naive Bayes Classification

- Sub-field of predictive modelling → **Supervised pattern classification**
 - Train model using labelled training data to assign pre-defined class label to new object
- **Naive Bayes Classifier** → Based on Bayes probability theorem



Good for document classification + disease predictions

- Linear classifier → Simple yet efficient
- 'Naive' assumption that features in dataset are mutually independent → **in practice, this is often violated, but it still performs under unrealistic assumptions**



For small sample sizes, it can outperform more powerful alternatives

- Strong violations of independence assumptions and non-linear classification problems → This will result in very poor performance

Posterior Probabilities

- Bayes' Rule → Thomas Bayes, probability model:

$$\text{posterior probability} = \frac{\text{conditional probability} \cdot \text{prior probability}}{\text{evidence}}$$

“What is the probability that the object belongs to class i ”

- Posterior Probability:

$$P(w_j|x_i) = \frac{P(x_i|w_j) \cdot P(w_j)}{P(x_i)}$$

- Objective is to maximise posterior probability, with training data to form decision rule

Class-Conditional Probabilities

- Assumption that Bayes classifiers make
 - Samples are *i.i.d* (Independent and identically distributed)
 - Conditional independence of features
 - Under *naive* assumption → class-conditional probabilities (likelihood), estimates are directly from training data

$$P(x|w_j) = P(x_1|w_j) \cdot P(x_2|w_j) \cdot \dots \cdot P(x_d|w_j) = \prod_{K=1}^d P(x_k|w_j)$$

- $P(x|w_j) \rightarrow$ How likely is it to observe particular pattern x given x belongs to w'_j
- ‘Individual’ likelihood → Estimated via maximum-likelihood (frequency for categorical data):

$$\hat{P}(x_i|w_j) = \frac{N_{x_i,w_j}}{n_{w_j}}, \quad (i = (1, \dots, d))$$

- N_{x_i,w_j} = Number of times x_i in w_j
- N_{w_j} = Total features count in w_j
- **Example:** 500 documents, where 100 documents are spam → Calculate class-conditional probability for new message ‘hello world’ given it’s spam

$$P(x = [\text{hello}, \text{world}]|w = \text{spam}) = P(\text{hello}|\text{spam}) \cdot P(\text{world}|\text{spam})$$

- **Maximum-likelihood estimate:**

$$\hat{P}(x = [hello, world] | w = spam) = \frac{20}{100} \cdot \frac{2}{100} = 0.004$$

- **Respect to naive assumption of conditional independence, we have a problem:**
 - Naive assumption - particular word does not influence the change of encountering other words in same document

Prior Probabilities

- Context of pattern classification, prior probabilities called *class priors* → The general probability of encountering a particular class
- Case of spam classification:

$P(spam)$ = "The probability that any new message is a spam message"

$$P(ham) = 1 - P(spam)$$

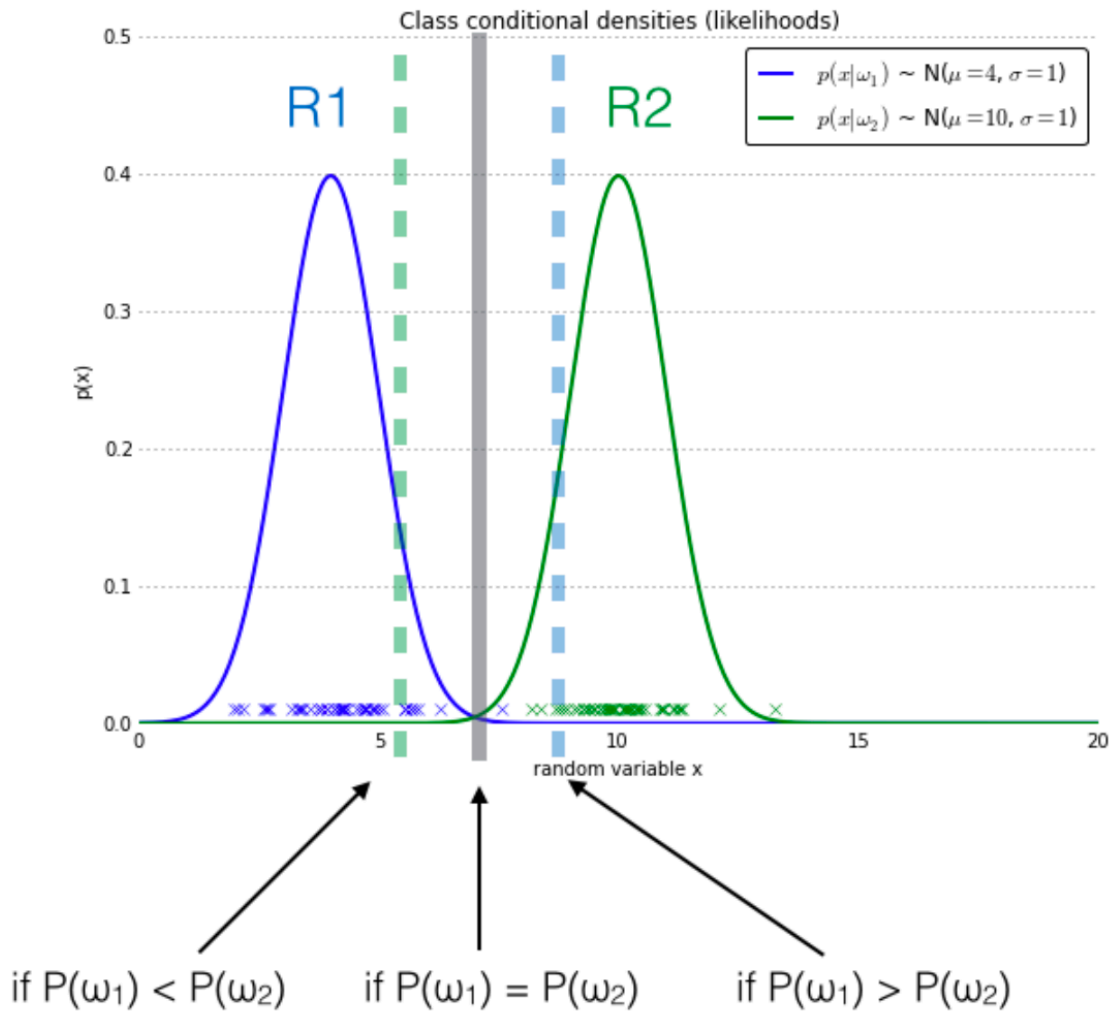
- If priors are following uniform distribution → posterior probabilities will entirely determine class-conditional probabilities and evidence term

$$\hat{P}(w_j) = \frac{N_{w_j}}{N_c}$$

- N_{w_j} = Sample count from class w_j
- N_c = Count of all samples

$$\hat{P}(spam) = \frac{\text{Number of spam messages in training data}}{\text{Number of all messages in training data}}$$

- **In the context of spam classification**



The effect of prior probabilities on the decision regions.

- Bell-curve denotes the probability densities of samples that were drawn from the two different normal distribution

Evidence

- Evidence - $P(x) \rightarrow$ probability of encountering a particular pattern x independent from the class label
- Given posterior probability is:

$$P(w_j|x_i) = \frac{P(x_i|w_j) \cdot P(w_j)}{P(x_i)}$$

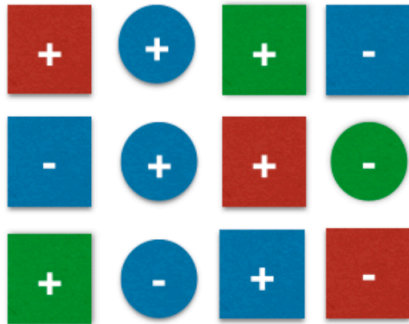
- Evidence:

$$P(x_i) = P(x_i|w_j) \cdot P(w_j) + P(x_i|w_j^C) \cdot P(w_j^C)$$

- w_j^C = Not class w_j

Multinomial Naive Bayes

- Toy Example



- w_j - Class label = $w_j \in \{+, -\}$
- x_i - 2D feature vector = $x_i = [x_{i1}, x_{i2}]$
 - $x_{i1} \in \{blue, green, red, yellow\}$
 - $x_{i2} \in \{circle, square\}$

- Can be written as:

$$x_i = [x_{i1}, x_{i2}]$$

for $i \in \{1, 2, \dots, n\}, n = 12$
and $x_{i1} \in \{blue, green, red, yellow\}$,
 $x_{i2} \in \{circle, square\}$

- Now the task is to classify new samples, pretending we don't know true label class '+'

Maximum-likelihood Estimates

- Decision rule:

Classify sample as + if

$$P(w = + | x = [blue, square]) \geq P(w = - | x = [blue, square])$$

else classify as -

- Under assumptions that samples are *i.i.d* → Prior probabilities obtained by maximum-likelihood



If samples are not *i.i.d* then MLE for prior probabilities would likely be biased and inaccurate

$$P(+) = \frac{7}{12} = 0.58$$

$$P(-) = \frac{5}{12} = 0.42$$

- Under 'naive' assumption that features 'colour' and 'shape' are mutually independent, class-conditional probabilities calculated as dot product

$$P(x|+) = P(blue|+) \cdot P(square|+) = \frac{3}{7} \cdot \frac{5}{7} = 0.31$$

$$P(x|-) = P(blue|-) \cdot P(square|-) = \frac{3}{5} \cdot \frac{3}{5} = 0.36$$

- Posterior probabilities = dot product of class-conditional and prior probabilities:

$$P(+|x) = P(x|+) \cdot P(+) = 0.31 \cdot 0.58 = 0.18$$

$$P(-|x) = P(x|-) \cdot P(-) = 0.36 \cdot 0.42 = 0.15$$

Classification

- New samples classified by plugging into posterior probabilities into decision rule:

If $P(+|x) \geq P(-|x)$
 classify as +
 else classify as -

- Since $0.18 > 0.15 \rightarrow$ sample classified as +

Additive Smoothing

- New examples $\rightarrow$ 'new' value $\rightarrow x = [yellow, square]$
- If yellow square not in training dataset, class-conditional and posterior probability = 0
 - To avoid this = smoothing term added with multinomial bayes
 - Most common:
 1. Lidstone Smoothing ($\alpha < 1$)
 2. Laplace Smoothing ($\alpha = 1$)

$$\hat{P}(x_i|w_j) = \frac{N_{x_i,w_j} + \alpha}{n_{w_j} + \alpha d}, \quad (i = (1, \dots, d))$$

- N_{x_i,w_j} = Number of times x_i in w_j
- N_{w_j} = Total feature count of w_j
- α = Additive smoothing parameter
- d = Dimensionality of feature vector $x = [x_1, \dots, x_d]$

Naive Bayes for Text Classification

The information is similar to what was covered in week 8 (can look at that besides what's covered here)

Bag-of-Words Model

- **Three main criteria of good features:**
 - **Salient** - Features are important and meaningful with respect to problem domain
 - **Invariant** - Features insusceptible to distortion, scaling, orientation, and more
 - **Discriminatory** - Features bare enough info → distinguish well between patterns
- Before fitting model → **represent text document as feature vector**
 - Bag-of-Words → **collection of different words that occur in training set + each word associated with count of occurrence**
- Vocab used to construct d-dimensional feature vector for each document → **dimensionality = number of different words in vocab** → **called vectorisation**

Tokenization

- **Breaking down text corpus into individual elements** → inputs for various NLP algorithms
- **Usually accompanied by other optional processing steps** → e.g. stop words + punctuation removal, stemming, lemmatizing, constructing n-grams

Stop words

- Stop words → common words in text → **considered un-informative (e.g. and, or, the, ...)**
- Approaches:
 - One → search against language-specific stop word dictionary
 - Another → Create **stop list** by sorting all words in entire text corpus by frequency

Stemming and Lemmatization

- **Stemming** → Transforming a word into root form → Martin F. Porter (1979) → **Porter stemmer**
 - Can create non-real words → e.g. 'thu' instead of 'thus'
- **Lemmatization** → Obtain canonical forms of words, called lemmas
 - Computationally more difficult and expensive over stemming

N-grams

- Token defined as sequence of n items
- Unigram (1-gram), bigram (2-gram), trigram (3-gram)
- Each word consists of one word/letter/symbol
- n depends on language and application:
 - English text book, $4 \leq n \leq 8$
 - Serbian text document, $3 \leq n \leq 7$ → Andelka Zecevic
 - Anti-spam email filtering, $n = 3$ and 4 → Kanaris e. al.

Decision Rule for Spam Classification

- Decision rule → posterior probabilities:

$$P(x|w = \text{spam}) \geq P(x|w = \text{ham})$$

- Posterior probabilities = product of class-conditional + prior probability:

$$\begin{aligned} P(x|w = \text{spam}) &= P(w = \text{spam}|x) \cdot P(\text{spam}) \\ P(x|w = \text{ham}) &= P(w = \text{ham}|x) \cdot P(\text{ham}) \end{aligned}$$

- Prior probabilities obtained via maximum-likelihood estimates:

$$\hat{P}(w = \text{spam}) = \frac{\text{Number of spam messages}}{\text{Number of all messages}}$$

$$\hat{P}(w = \text{ham}) = \frac{\text{Number of ham messages}}{\text{Number of all messages}}$$

- Assuming words in each document are conditionally independent. → Two models to compute class-conditional probabilities:
 - Multi-variate Bernoulli model

- Multinomial model

Multi-variate Bernoulli Naive Bayes

- **Based on binary data** → Every token in feature vector is either 1 or 0
- Feature vector has m dimensions → m = Number of words in whole vocabulary.
- 1 means word occurs in doc, 0 means it doesn't
- Bernoulli written as:

$$P(x|w_j) = \prod_{i=1}^m P(x_i|w_j)^b \cdot (1 - P(x_i|w_j))^{(1-b)} \quad (b \in \{0, 1\})$$

- $\hat{P}(x|w_j)$ = Maximum-likelihood → Estimate particular word (or token) x_i occurs in w_j :

$$\hat{P}(x_i|w_j) = \frac{df_{xi,y} + 1}{df_y + 2}$$

- $df_{xi,y}$ = Number of documents in training dataset that contain x_i in w_j
- df_y = Number of documents in training dataset → belongs to w_j
- $+1, +2$ = Laplace smoothing parameters

Multinomial Bayes

Term Frequency

- Alternative approach to characterise text documents.
- **Term Frequency ($tf(t, d)$)** → Defined as number of times t (word/token) in d (documents) → sometimes called **raw frequencies**
 - Normalised by dividing raw term frequency with document length

$$\text{Normalised term frequency} = \frac{tf(t, d)}{n_d}$$

- $tf(t, d)$ = Raw term frequency (count t in d)
- n_d = total number of terms in document d

- Term frequency then used to compute maximum-likelihood:

$$\hat{P}(x_i|w_j) = \frac{\sum tf(x_i, d \in w_j) + \alpha}{\sum N_{d \in w_j} + \alpha \cdot V}$$

- x_i = Word from feature vector x of particular sample
- $\sum tf(x_i, d \in w_j)$ = Sum of rwa term frequencies of word x_i , from all documents in training sample class w_j
- $\sum N_{d \in w_j}$ = Sum of all term frequencies in training datasets for class w_j
- α = Additive smoothing parameter
- V = Size of vocab
- Class-conditional probability of encountering text $x \rightarrow$ calculated as product from likelihoods of individual words

$$P(x|w_j) = P(x_1|w_j) \cdot P(x_2|w_j) \cdot \dots \cdot P(x_n|w_j) = \prod_{i=1}^m P(x_i|w_j)$$

Term Frequency - Inverse Document Frequency (tf-idf)

- Another alternative for characterising text documents
- **Can be understood as weighted term frequency** $\rightarrow$ useful for if stop words have not been removed
- Tf-idf assumes $\rightarrow$ importance of a word is inversely proportional to how often it occurs across all documents
- Tf-idf $\rightarrow$ Most commonly used to rank documents by relevance in different text mining tasks $\rightarrow$ e.g. Page ranking for search engines
- Also applicable to text classification via Naive Bayes:

$$\text{tf-idf} = tf(t, d) \cdot idf(t)$$

- Tf-idf - normalised term frequency, idf - inverse document frequency:

$$idf(t) = \log \frac{n_d}{n_d^{(t)}}$$

- n_d = Total number of documents
- $n_d^{(t)}$ = Numer of documents that contain term t

Performances of Multi-Variate Bernoulli and Multinomial Model



Multinomial > Multi-variate Bernoulli

- If vocab size is relatively small



Performance of machine learning algorithm dependant on feature choice

Naive Bayes Variants

Gaussian Naive Bayes

- Naive Bayes can also be used on continuous data
- Iris flow dataset → good example of continuous features
 - Strategy → Discretise features and form categories or use gaussian kernel
- Gaussian Naive Bayes:
 - Assumption is that probability distributions of features follow normal gaussian distribution

$$P(x_i|w) = \frac{1}{\sqrt{2\pi\sigma_w^2}} \exp\left(-\frac{(x_{ik} - \mu_w)^2}{2\sigma_w^2}\right)$$

- μ = Sample mean
 - σ = Standard deviation
- Naive bayes assumption of conditional independence → Class-conditional probabilities computed as product of individual probabilities:

$$P(x_i|w) = \prod_{k=1}^d P(x_{ik}|w)$$

Eager and Lazy Learning Algorithms

- **Eager learners** → Fast with classifying new instances
 - Learn a model from training dataset as soon as data is available

- Once model learned → no re-evaluation needed for training data to make new predictions
- Computationally most expensive → model build steps
- **Lazy learners** → Memorise and re-evaluate training dataset for predicting class labels
 - Advantage → Model building (training) is fast
 - But prediction is slow → because of re-evaluation
 - Disadvantage → Training data retained → Expensive for storage
 - Example of lazy learner → KNN